Naveen Sellathurai
33106770

Declaration

I, [Student Name], declare that this assignment, titled A1.4: Research Report, is my own original work and has not been copied from any other source except where explicitly acknowledged. I have not engaged in plagiarism, collusion, or any other form of academic misconduct in the preparation and submission of this assignment. All sources of information and data used in this assignment have been properly cited and referenced in accordance with the prescribed guidelines. I have not used unauthorized assistance in the preparation of this assignment and have not allowed any other student to copy my work. I am aware that any breach of academic integrity may result in disciplinary action as per the policies of Monash University, which may include failing this assignment or the course, and further academic penalties.

Signature: Naveen Sellathurai Date: 02/10/2026

Self-Evaluation

Rate your performance for each criteria. Put a ✅ (tick) in the box where you think your work belongs.

Criteria	Exceeds Expectations	Meets Expectations	Needs Improvement	Fail to meet expectations
Introduction		✅		
Main Body		✅		
Reflection		✅		
AI Acknowledgement		✅		

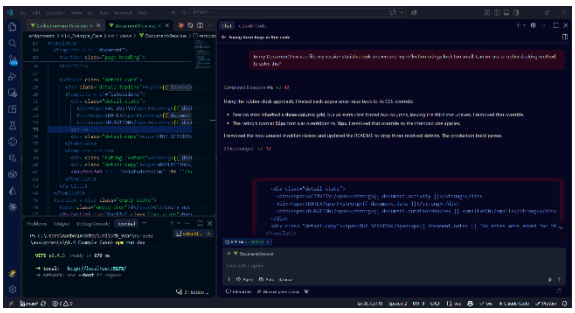
Naveen Sellathurai
33106770

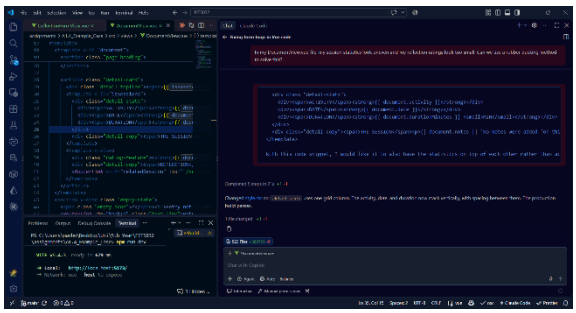
Section 1: Introduction

Debugging is a vital skill to know when it comes to development, and this is the same case with JavaScript. With AI Technology rising, developers will continue to use generative AI to help building up their features, but when this happens, how will they understand the code when it breaks. If they use AI to assist in production, then they will also use GenAI or Agentic AI to help with JavaScript debugging. As a result, we need to consider the different strategies there are to help effectively use AI for debugging.

Section 2: Main Body

One strategy for debugging JavaScript code is by implementing a 'Rubber Ducking' method with GenAI. This is the idea of treating the agent like a person in a conversation and explaining the problem step by step via multiple prompts [1]. For example, if there's a specific feature that is not functioning, instead of just asking a GenAI tool like ChatGPT to solve the issue, a developer can query different questions that can lead to them identifying the solution themselves by debugging with a the GenAI Agent, like a teacher-student relationship.





This strategy is excellent with getting the GenAI to understand the context of a feature and not provide responses that are irrelevant to the context of the feature and thus provide a solution that satisfies its requirements. This method however does require more time and effort since it involves breaking down the problem bit by bit, which is counterintuitive to developers who use GenAI with generalised prompts.

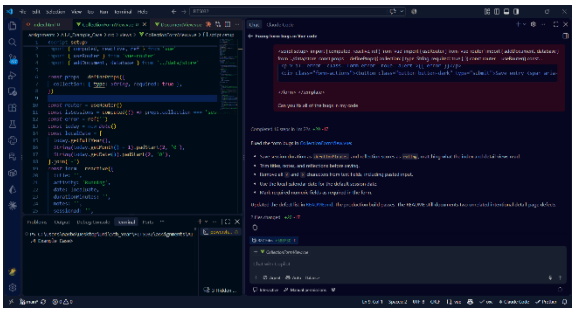
Looking more into testing, GenAI can also be used to help with debugging by creating Unit Tests for features. This can investigate negative testing and testing for edge-cases [2]. Having unit tests created with GenAI significantly saves time due to how much time it takes to create unit tests to cover every possible case. However, whilst this can help with bug fixing for errors involving calculations and input sanitisation, using GenAI to develop unit tests does not actually help with debugging to find out why bugs that are caused by code not matching the feature's context are happening and how they can be fixed.

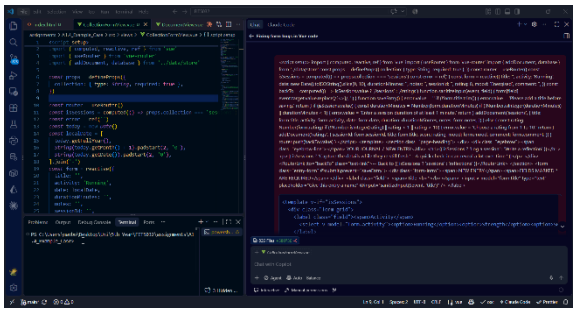
Another strategy can also consider dynamic testing instead of static testing by using Agentic AI to have it constantly run the JavaScript application independently to undergo debugging and error catching as coding happens. For example, in a Vue.js application, as the developer builds up new features, they can ask Agentic AI to analyse for any potential bugs by running the current application in its own environment [3]. The primary benefit of this is that because it's automated and dynamic, it does not require the developer to manually input any prompts, pieces of code or error messages. However, this

is also extremely expensive. Doing dynamic testing would include running Agentic AI over long periods of time, which would build up large bills based on the model being used.

When developers receive errors in their code when they try running it, GenAI can be used to analyse the error logs and break down what they mean and what exactly the error is that is causing the error, alongside how to fix it. When the error happens, the developer simply copy pastes the entire error log into the GenAI tool so that it can analyse it and explain what the error is and how to fix it. This makes it an easy strategy to use, however because the developer is just copy-pasting the error log into the prompt and telling it to fix the issue, it can more easily result in hallucinations as the developer doesn't actually provide context for what the feature is supposed to be doing it, and therefore even if the GenAI tool fixes the error, it may provide a solution that produces an unwanted output compared to what requirements want.

Similarly, a developer can also dump their code for a feature into a GenAI tool and ask it to bugfix it entirely. This strategy is the easiest one with minimal effort, but this also means it will produce the most hallucinations. This method is especially used when a single file contains multiple errors, and to cut down time, developers just send the entire code to a GenAI tool to fix every error at once and clean up the code.

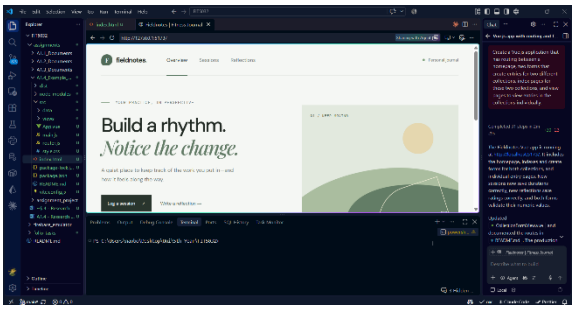


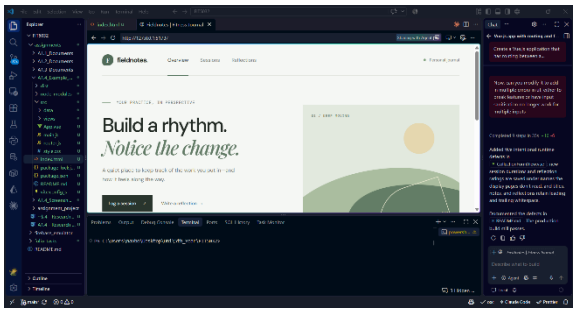


A benefit is that this also keeps the entire code consistent to coding practices and fix any formatting errors and is usually quick compared to Rubber Ducking. Due to it being more prone to hallucinating, however, it can end up producing more bugs whilst cleaning up current bugs, and there is also a security risk by putting the entire code block into the GenAI Tool.

Section 3: Reflection

To test some of these strategies, GenAI was used to build a small Vue.js application, deliberately adding in errors into the code so that it can be used to build my findings.





When I tried the dumping strategy, it analysed my entire code in the file and then completed every fix in the one response. Results proved to have no hallucinations after manually testing the fixes.

When I used the rubber ducking strategy, I was able to explain what I wanted to fix in terms of formatting my frontend in multiple steps to receive the fixes that I wanted. It understood these requests and made the changes.

At a small scale, GenAI can be used to fix bugs with minimal issues, but at a larger scale, it can be more troublesome. If overused, this can ethically lead to developers not manually testing their own work and assume that AI can fix all of their bugs and errors.

Section 4: Conclusion

At a small scale, using GenAI and AgenticAI can be an excellent tool for debugging, but for larger scale projects, it is ideal to use these tools in small blocks and break down needs, rather than giving these tools the entire codebase to bugfix on its own. Further research and development should be done with by testing these strategies on a larger scale project.

Section 5: Acknowledgement of AI use

I used AI in the following ways:

- Creating a small Vue.js application for the purpose of showcasing a debugging process using GenAI
- Running prompts with GenAI for the debugging process

In both cases above, I used Github Copilot, VS Code's Built-in AI tool, to first build the small basic Vue.js application and add to deliberately errors and bugs

References

[1] "The Rubber Duck Method With AI for Faster Bug Fixes," Vibe Coder Blog. https://blog.vibecoder.me/rubber-duck-method-ai-explaining-bugs (accessed Oct. 2, 2026).

[2] Codecademy Team, "How to use Generative AI for Unit Testing," Codecademy. https://www.codecademy.com/article/create-unit-tests-using-generative-ai (accessed Oct. 2, 2026).

[3] "Agentic AI: A complete guide," Pegasystems. https://www.pega.com/agentic-ai (accessed Oct. 2, 2026).